Estudio de Magic Utilities 2006 v4.22

Programa:	Magic Utilities 2006 v4.22	
Protección:	Empakado con PECompact v2.x + Nagscreen + Trial x días	
Objetivos:	Desempakar y evitar las limitaciones	
URL descarga:	http://www.magictweak.com	
Dificultad:	bastante sencilla	
Herramientas:	OllyDbg v1.10 + PEiD v0.92 + ImpREC v1.6	
Cracker:	SyXe'05 .::[hAcK-c0d3d]..[CracksLatinoS]::.	Fecha: 17-02-06

Un saludo a todos CracksLatinoS!

Bueno hoy vamos a vernos las caras con un esquema de protección que cada vez más nos solemos encontrar en la industria Shareware y se trata de un TimeTrial + Nagscreen al que se le agrega un regalito que después vamos a desenvolver, todo a su debido tiempo. De momento nos limitamos a instalar la app Magic Utilities 2006 4.22 en nuestro querido PC, después analizamos su ejecutable mgutil.exe a ver qué nos dice PEiD:

PECompact 2.x -> Jeremy Collake

vaya PECompact 2.x, deberemos leernos un poco la parte 31 de la estupenda introducción al cracking en Ollydbg de Ricardo, jeje.. ahora que estamos con los packers nos vendrá bien un poco de práctica y este espero que sea una buena pieza de inicio. Intentaré mostrar un poco por encima lo que hace este compresor, no sé si estará bien llamaré packer porque su único objetivo es comprimir el código del ejecutable nada más, no modifica el EP (stolen bytes) ni redirige la IAT (la deja intacta), aún así hace algo que me parece original y que comentaré a continuación ;-)

Cerramos PEiD y nos abrimos un Ollydbg con el plugin IsDebuggerPresent que nos lo oculta automáticamente cambiando el byte [7FFDF002] bajo WinXP que es [EBX+2] en el EP. Cargamos mgutil.exe y quedamos detenidos en su Entry Point:

```
Address   Hex dump        Disassembly                          Comment
0045C169  B8 D4F85600     MOV EAX,mgutil.0056F8D4
0045C16E  50              PUSH EAX
0045C16F  64:FF35 000000( PUSH DWORD PTR FS:[0]
0045C176  64:8925 000000( MOV DWORD PTR FS:[0],ESP
0045C17D  33C0            XOR EAX,EAX
0045C17F  8908            MOV DWORD PTR DS:[EAX],ECX  <---  Cuidado aquí!!
0045C181  50              PUSH EAX
0045C182  45              INC EBP
0045C183  43              INC EBX
0045C184  6F              OUTS DX,DWORD PTR ES:[EDI]          I/O command
0045C185  6D              INS DWORD PTR ES:[EDI],DX           I/O command
0045C186  v 70 61         JO SHORT mgutil.0045C1E9
0045C188  637432 00       ARPL WORD PTR DS:[EDX+ESI],SI
0045C18C  12D9            ADC BL,CL
0045C18E  6E              OUTS DX,BYTE PTR ES:[EDI]           I/O command
0045C18F  CB              RETF                                Far return
```

Allí estamos parados, en las cuatro primeras instrucciones lo que hará será instalar su propio manejador de excepciones en :0056F8D4 para luego levantar una excepción en :0045C17F a propósito para comenzar a hacer de las suyas. Primero veamos como deja el manejador instalado (ver el stack)

```
0012FFBC  0012FFE0  Pointer to next SEH record
0012FFC0  0056F8D4  SE handler
0012FFC4  77E614C7  RETURN to kernel32.77E614C7
0012FFC8  00000000
0012FFCC  00000000
```

Ya lo vemos allí, lo siguiente que hace es poner EAX a cero y levantar la excepción que nos llevará al SEH recién instalado, para curarnos en salud vamos antes a él y le ponemos un BP y después dejamos que la cree para caer allí:

```
Address   Hex dump    Disassembly                          Comment
0056F8D4  B8 7DE8CAFF MOV EAX,FFCAE87D
0056F8D9  8D88 7A108C00 LEA ECX,DWORD PTR DS:[EAX+8C107A]
0056F8DF  8941 01     MOV DWORD PTR DS:[ECX+1],EAX
0056F8E2  8B5424 04   MOV EDX,DWORD PTR SS:[ESP+4]
0056F8E6  8B52 0C     MOV EDX,DWORD PTR DS:[EDX+C]
0056F8E9  C602 E9     MOV BYTE PTR DS:[EDX],0E9            Inserta un JMP
0056F8EC  83C2 05     ADD EDX,5
0056F8EF  2BCA        SUB ECX,EDX
0056F8F1  894A FC     MOV DWORD PTR DS:[EDX-4],ECX         Y lo reajusta
0056F8F4  33C0        XOR EAX,EAX
0056F8F6  C3          RETN
```

Como vemos no es un SEH común pues ni si quiera se para a mirar qué tipo de excepción se ha generado, lo único que hace es insertar un JMP justo después del EP, jeje… supongo que lo hace así para intentar despistar al Cracker, traceamos hasta el RETN y vemos como efectivamente insertó un JMP allí:

```
Address   Hex dump    Disassembly                          Comment
0045C169  B8 D4F85600 MOV EAX,mgutil.0056F8D4
0045C16E  50          PUSH EAX
0045C16F  64:FF35 000000( PUSH DWORD PTR FS:[0]
0045C176  64:8925 000000( MOV DWORD PTR FS:[0],ESP
0045C17D  33C0        XOR EAX,EAX
0045C17F - E9 73371100 JMP mgutil.0056F8F7
0045C184  6F          OUTS DX,DWORD PTR ES:[EDI]           I/O command
0045C185  6D          INS DWORD PTR ES:[EDI],DX            I/O command
0045C186 v 70 61      JO SHORT mgutil.0045C1E9
0045C188  637432 00   ARPL WORD PTR DS:[EDX+ESI],SI
```

Y apunta a :56F8F7 de momento traceamos el SEH hasta el RETN y lo superamos para caer en el módulo ntdll:

```
Address   Hex dump    Disassembly                          Comment
77F69BA4  64:8B25 000000( MOV ESP,DWORD PTR FS:[0]
77F69BAB  64:8F05 000000( POP DWORD PTR FS:[0]
77F69BB2  8BE5        MOV ESP,EBP
77F69BB4  5D          POP EBP
77F69BB5  C2 1400     RETN 14
77F69BB8  8B4C24 04   MOV ECX,DWORD PTR SS:[ESP+4]
77F69BBC  F741 04 060000( TEST DWORD PTR DS:[ECX+4],6
```

Para no matarnos traceando hasta que llame a ntdll.ZwContinue lo que haremos será poner un BPM en la sección de código del programa:

```
Address  Size     Owner    Section  Contains   Type Access Initial Mapped as
003E0000 00001000                              Priv RW     RW
003F0000 00008000                              Priv RW     RW
00400000 00001000 mgutil            PE header  Imag R      RWE
00401000 00151000 mgutil   .text    code       Imag R      RWE
00552000 000AF000 mgutil   .rsrc    impo
00610000 00001000 ole32             PE he
00611000 00104000 ole32    .text    code,
00715000 00006000 ole32    .orpc    code
0071B000 00007000 ole32    .data    data
00722000 00002000 ole32    .rsrc    resou
00724000 0000D000 ole32    .reloc   reloc
00740000 00007000
00800000 00002000
00810000 00103000
00920000 000E8000
00C20000 00008000
00D20000 00001000
00DA0000 00004000
00DB0000 00003000
00DC0000 00002000
015B0000 00002000
10000000 00001000 LIBMYSQL          PE he
10001000 0001C000 LIBMYSQL .text    code
1001D000 00004000 LIBMYSQL .rdata   impo
10021000 00020000 LIBMYSQL .data    data
10041000 00002000 LIBMYSQL .reloc   reloc
5F1F0000 00001000 OLEPRO32          PE he
5F1F1000 00012000 OLEPRO32 .text    code,
5F203000 00001000 OLEPRO32 .data    data
5F204000 00004000 OLEPRO32 .rsrc    resou
5F208000 00002000 OLEPRO32 .reloc   relocations Imag R      RWE
```

```
Actualize
View in Disassembler                Enter
Dump in CPU
Dump
Search                              Ctrl+B
Set break-on-access                 F2
Set memory breakpoint on access
Set memory breakpoint on write
Set access                          ▸
Copy to clipboard                   ▸
Sort by                             ▸
Appearance                          ▸
```

Y pulsamos F9… al momento caemos en ella:

Address	Hex dump	Disassembly	Comment
0045C17F	- E9 73371100	JMP mgutil.0056F8F7	
0045C184	6F	OUTS DX,DWORD PTR ES:[EDI]	I/O command
0045C185	6D	INS DWORD PTR ES:[EDI],DX	I/O command
0045C186	v 70 61	JO SHORT mgutil.0045C1E9	
0045C188	637432 00	ARPL WORD PTR DS:[EDX+ESI],SI	
0045C18C	12D9	ADC BL,CL	

Mira por donde nos deja justo en el JMP insertado, jeje.. tanto rollo para esto, intentamos seguir el salto con F7 y qué???????

Nos ha activado el flag T (Trap flag) que si está activado no nos deja tracear, veamos:

```
C 0  ES 0023 32bit 0(FFFFFFFF)
P 1  CS 001B 32bit 0(FFFFFFFF)
A 0  SS 0023 32bit 0(FFFFFFFF)
Z 1  DS 0023 32bit 0(FFFFFFFF)
S 0  FS 0038 32bit 7FFDE000(FFF)
T 1  GS 0000 NULL
D 0
O 0  LastErr ERROR_SUCCESS (00000000)
EFL 00010346 (NO,NB,E,BE,NS,PE,GE,LE)
```

Lo que haremos será simplemente desactivarlo así sin más, situamos el puntero sobre él y le hacemos doble clic, automáticamente volverá a su valor original T==0 y nos volverá a dejar tracear el código, apretamos F7 para seguir el JMP y caemos en un código algo más alegre, jejeje… veámoslo:

Address	Hex dump	Disassembly	Comment
0056F8F7	B8 7DE8CAFF	MOV EAX,FFCAE87D	
0056F8FC	64:8F05 000000	POP DWORD PTR FS:[0]	
0056F903	83C4 04	ADD ESP,4	
0056F906	55	PUSH EBP	
0056F907	53	PUSH EBX	
0056F908	51	PUSH ECX	
0056F909	57	PUSH EDI	
0056F90A	56	PUSH ESI	
0056F90B	8D98 33108C00	LEA EBX,DWORD PTR DS:[EAX+8C1033]	
0056F911	8B53 18	MOV EDX,DWORD PTR DS:[EBX+18]	
0056F914	8BE8	MOV EBP,EAX	
0056F916	6A 40	PUSH 40	
0056F918	68 00100000	PUSH 1000	
0056F91D	FF73 04	PUSH DWORD PTR DS:[EBX+4]	
0056F920	6A 00	PUSH 0	
0056F922	8B4B 10	MOV ECX,DWORD PTR DS:[EBX+10]	
0056F925	03CA	ADD ECX,EDX	
0056F927	8B01	MOV EAX,DWORD PTR DS:[ECX]	
0056F929	FFD0	CALL EAX	
0056F92B	8BF8	MOV EDI,EAX	
0056F92D	50	PUSH EAX	
0056F92E	8B33	MOV ESI,DWORD PTR DS:[EBX]	
0056F930	8B53 18	MOV EDX,DWORD PTR DS:[EBX+18]	
0056F933	8B43 20	MOV EAX,DWORD PTR DS:[EBX+20]	

Si llegamos a la altura del CALL EAX veremos que llamará a VirtualAlloc, sabemos que esta api se utiliza para solicitar/reservar memoria, en este caso (y en mi máquina) el retorno queda EAX==00DD0000 lo que significa que ya creó este bloque de memoria:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
00DB0000	00003000				Priv	RW	RW	
00DC0000	00002000				Map	R	R	
00DD0000	00001000				Priv	RWE	RWE	
015B0000	00002000				Map	R	R	
10000000	00001000	LIBMYSQL		PE header	Imag	R	RWE	
10001000	0001C000	LIBMYSQL	.text	code	Imag	R	RWE	
1001D000	00004000	LIBMYSQL	.rdata	imports,exp	Imag	R	RWE	
10021000	00020000	LIBMYSQL	.data	data	Imag	R	RWE	
10041000	00002000	LIBMYSQL	.reloc	relocations	Imag	R	RWE	

Allí lo vemos, pues bien esta sección la utilizará para desempakar el código del ejecutable, justo en esta llamada:

Address	Hex dump	Disassembly	Comment
0056F981	51	PUSH ECX	
0056F982	894E 28	MOV DWORD PTR DS:[ESI+28],ECX	
0056F985	FFD7	CALL EDI	desempaka el codigo..
0056F987	8985 2D118C00	MOV DWORD PTR SS:[EBP+8C112D],EAX	
0056F98D	8BF0	MOV ESI,EAX	
0056F98F	59	POP ECX	
0056F990	034B 18	ADD ECX,DWORD PTR DS:[EBX+18]	
0056F993	68 00800000	PUSH 8000	
0056F998	6A 00	PUSH 0	
0056F99A	57	PUSH EDI	
0056F99B	FF11	CALL DWORD PTR DS:[ECX]	
0056F99D	8BC6	MOV EAX,ESI	
0056F99F	5E	POP ESI	
0056F9A0	5F	POP EDI	
0056F9A1	59	POP ECX	
0056F9A2	5B	POP EBX	
0056F9A3	5D	POP EBP	
0056F9A4	FFE0	JMP EAX	jejeje...
0056F9A6	1C 0B	SBB AL,0B	
0056F9A8	0000	ADD BYTE PTR DS:[EAX],AL	
0056F9AA	0000	ADD BYTE PTR DS:[EAX],AL	
0056F9AC	0000	ADD BYTE PTR DS:[EAX],AL	

Si nos fijamos en la ventana de registros vemos EDI==00DD0218 que es donde apunta la llamada, dentro de la misma hace otra llamada a VirtualAlloc, le asigna el bloque 00DE0000 que luego libera al terminar:

Address	Hex dump	Disassembly	Comment
00DD0398	8B7E 08	MOV EDI,DWORD PTR DS:[ESI+8]	
00DD039B	03C7	ADD EAX,EDI	
00DD039D	FF10	CALL DWORD PTR DS:[EAX]	kernel32.VirtualFree
00DD039F	8B46 0C	MOV EAX,DWORD PTR DS:[ESI+C]	
00DD03A2	03C7	ADD EAX,EDI	
00DD03A4	5D	POP EBP	
00DD03A5	5E	POP ESI	
00DD03A6	5F	POP EDI	
00DD03A7	59	POP ECX	
00DD03A8	5B	POP EBX	
00DD03A9	C3	RETN	

Al llegar al RETN salimos justo después del CALL en el que entramos:

Address	Hex dump	Disassembly	Comment
0056F97E	8B4B 14	MOV ECX,DWORD PTR DS:[EBX+14]	
0056F981	51	PUSH ECX	
0056F982	894E 28	MOV DWORD PTR DS:[ESI+28],ECX	
0056F985	FFD7	CALL EDI	desempaka el codigo..
0056F987	8985 2D118C00	MOV DWORD PTR SS:[EBP+8C112D],EAX	mgutil.<ModuleEntryPoint>
0056F98D	8BF0	MOV ESI,EAX	
0056F98F	59	POP ECX	

En EAX lleva un valor importante, aunque veamos <ModuleEntryPoint> al lado y que en realidad es así, si vamos a esa dirección :45C169 vemos el OEP:

Address	Hex dump	Disassembly	Comment
0045C169	55	PUSH EBP	
0045C16A	8BEC	MOV EBP,ESP	
0045C16C	6A FF	PUSH -1	
0045C16E	68 B8434900	PUSH mgutil.004943B8	
0045C173	68 48254600	PUSH mgutil.00462548	
0045C178	64:A1 00000000	MOV EAX,DWORD PTR FS:[0]	
0045C17E	50	PUSH EAX	
0045C17F	64:8925 000000	MOV DWORD PTR FS:[0],ESP	
0045C186	83EC 58	SUB ESP,58	
0045C189	53	PUSH EBX	
0045C18A	56	PUSH ESI	
0045C18B	57	PUSH EDI	
0045C18C	8965 E8	MOV DWORD PTR SS:[EBP-18],ESP	
0045C18F	FF15 B0D24800	CALL DWORD PTR DS:[48D2B0]	kernel32.GetVersion
0045C195	33D2	XOR EDX,EDX	
0045C197	8AD4	MOV DL,AH	
0045C199	8915 58FD4A00	MOV DWORD PTR DS:[4AFD58],EDX	

Que casualmente coincide con el EntryPoint del packer, jeje.. recordemos que estaba allí mismo, lo que ocurre es que desempakó el código allí mismo sobrescribiendo el código inicial (del packer), bueno igual da esto, traceamos un poco con F8 y llegamos a otro "CALL DWORD PTR [ECX]" donde libera la primera sección :00DD0000 con VirtualFree, la superamos y enseguida llega el salto al OEP:

```
Address   Hex dump        Disassembly                    Comment
0056F998    6A 00         PUSH 0
0056F99A    57            PUSH EDI
0056F99B    FF11          CALL DWORD PTR DS:[ECX]        llama a VirtualFree!!
0056F99D    8BC6          MOV EAX,ESI
0056F99F    5E            POP ESI
0056F9A0    5F            POP EDI
0056F9A1    59            POP ECX
0056F9A2    5B            POP EBX
0056F9A3    5D            POP EBP
0056F9A4  - FFE0          JMP EAX                        mgutil.<ModuleEntryPoint>
0056F9A6    1C 0B         SBB AL,0B
0056F9A8    0000          ADD BYTE PTR DS:[EAX],AL
0056F9AA    69C1 45000000 IMUL EAX,ECX,45
0056F9B0    0000          ADD BYTE PTR DS:[EAX],AL
0056F9B2    0000          ADD BYTE PTR DS:[EAX],AL
```

Si lo seguimos caemos en el mismo OEP, desde aquí dumpearemos el archivo, el mismo OllyDump nos hará bien el trabajo, las opciones de siempre, nombramos el dumpeado como tute.exe así nos referimos al mismo archivo todos.

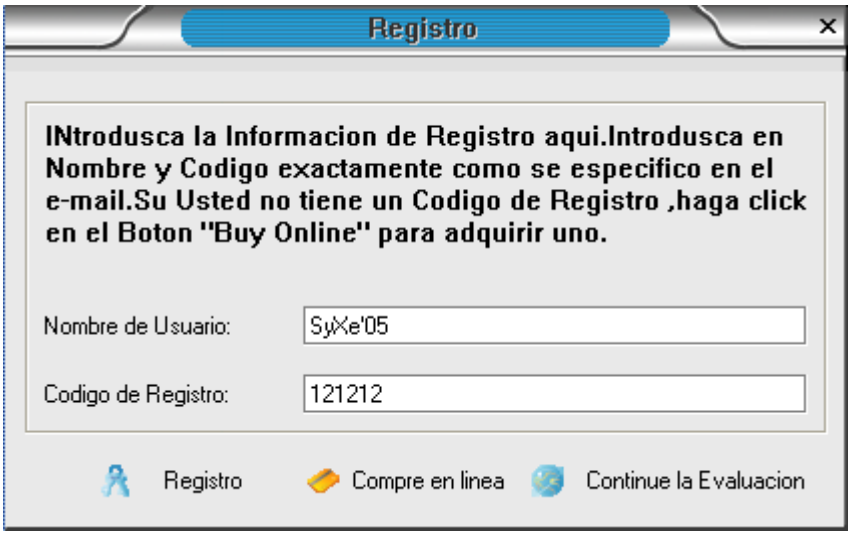
Acto seguido abrimos el ImpRec y attacheamos el proceso de la lista, le damos los valor correcto del OEP que será 5C169 (que en realidad ya está allí puesto dado que coincide con el EP) picamos en "IAT AutoSearch" y "Get Imports" nos aparecen todas las entradas correctas, sólo nos queda reparar el dumpeado tute.exe, elegimos "Fix Dump" y en el explorer que abre lo seleccionamos, lo repara sin problemas y nos informa de ello:

```
Fixing a dumped file...
10 (decimal:16) module(s)
1CA (decimal:458) imported function(s).
*** New section added successfully. RVA:00201000 SIZE:00003000
Image Import Descriptor size: 140; Total length: 20DE
C:\Archivos de programa\Mgutil 2006 4.22\tute_.exe saved successfully.
```

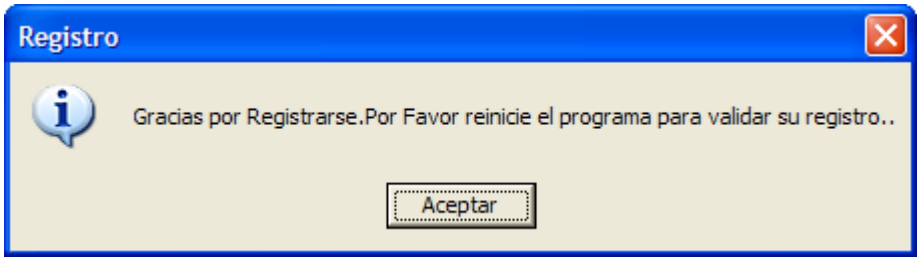
Nos ha dejado el archivo tute_.exe al lado y que debe funcionar, lo probamos y efectivamente funciona, jeje.. adiós PECompact, ahora que tenemos listo el desempakado nos centraremos en las limitaciones, los que me conocen saben que yo actúo así, siempre me gusta terminar el trabajo, si tiene limitaciones ¿porqué parar? Vamos a quitarlas

Superando las limitaciones…

Lo primero que hacemos es abrirnos el dumpeado tute_.exe en el Olly y ejecutarlo a ver como se comporta, lo primero que vemos es esta nag:

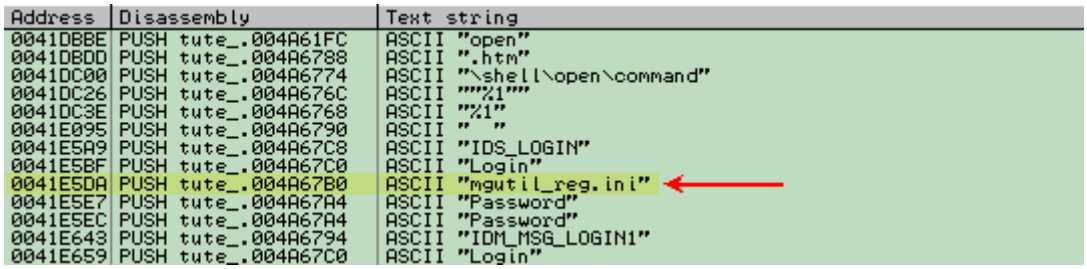


Ya le he puesto unos datos, lo que haremos será ver si tira mensaje de error, apretamos en el botón Registro y efectivamente muestra uno:



Aunque mucho no nos va a ayudar, nos dice que reiniciemos, lo vemos en español porque instale este lenguaje bajándolo de la misma página del programa, aceptamos el mensajito y nos cierra el programa:

Pulsamos Ctrl+F2 y lo volvemos a ejecutar, quedamos en el OEP, sabemos que va a validar los datos de registro en cualquier lugar, antes echamos una miradita en las strings:



Address	Disassembly	Text string
0041DBBE	PUSH tute_.004A61FC	ASCII "open"
0041DBDD	PUSH tute_.004A6788	ASCII ".htm"
0041DC00	PUSH tute_.004A6774	ASCII "\shell\open\command"
0041DC26	PUSH tute_.004A676C	ASCII ""%1""
0041DC3E	PUSH tute_.004A6768	ASCII "%1"
0041E095	PUSH tute_.004A6790	ASCII " "
0041E5A9	PUSH tute_.004A67C8	ASCII "IDS_LOGIN"
0041E5BF	PUSH tute_.004A67C0	ASCII "Login"
0041E5DA	PUSH tute_.004A67B0	ASCII "mgutil_reg.ini"
0041E5E7	PUSH tute_.004A67A4	ASCII "Password"
0041E5EC	PUSH tute_.004A67A4	ASCII "Password"
0041E643	PUSH tute_.004A6794	ASCII "IDM_MSG_LOGIN1"
0041E659	PUSH tute_.004A67C0	ASCII "Login"

Como parece ser que va a mirar en un archivo .ini pongo estos BPs antes de continuar:

Que se utilizan para leer cadenas y enteros de los archivos .ini respectivamente, refiriéndome a enteros como números "123" por ejemplo, si vamos al archivo donde tenemos instalado el Window$ allí veremos el nombre de dos archivos .ini como estos:

mgutil_reg.ini
mgutil_win.ini

Si le echamos un vistazo al segundo "mgutil_win.ini" veremos los datos que pusimos ;)

```
********************//
[REG]
Times=4
1=1140162652
2=1140162876
3=1140163275
4=1140163375
UserName=SyXe'05
RegCode=121212
********************//
```

O sea que los guardó en el archivo "mgutil_win.ini" vale reinicio Olly y lo ejecuto con F9 dejando puestos los dos breakpoints anteriores para ver cuando

trata de leer el archivo donde tiene almacenados los datos de registro, cuando para por primera vez miramos la pila (stack) para ver los parámetros pasados a la api:

```
Address   Value     Comment
00127898  00420E88  CALL to GetPrivateProfileStringA from tute_.00420E86
0012789C  004A67D4  Section = "Settings"
001278A0  004A6C2C  Key = "Language"
001278A4  004A6C38  Default = "English.ini"
001278A8  001279E0  ReturnBuffer = 001279E0
001278AC  00000028  BufSize = 28 (40.)
001278B0  004A67B0  IniFileName = "mgutil_reg.ini"
001278B4  004ACEF8  ASCII "-ÜH"
```

Va a leer del que no nos interesa pues no puso nada allí de nuestros datos sino que allí guarda el lenguaje, tipo de skin, y cosas por el estilo que no nos interesan, apretamos F9 y vuelve a parar en este archivo, después lee "skin.ini" situado en la carpeta Skins dentro del directorio de instalación del programa, seguimos apretando F9 hasta que pare donde nosotros queremos:

```
Address   Value     Comment
00127898  004214AA  CALL to GetPrivateProfileStringA from tute_.004214A8
0012789C  004A6A90  Section = "REG"
001278A0  004A6A94  Key = "UserName"
001278A4  004ACED4  Default = ""
001278A8  00127A08  ReturnBuffer = 00127A08
001278AC  00000028  BufSize = 28 (40.)
001278B0  004A6AA0  IniFileName = "mgutil_win.ini"
001278B4  004ACEF8  ASCII "-ÜH"
```

Comienza leyendo el nombre "UserName" que guardará en :00127A08 como vemos, apretamos nuevamente F9 y ahora sí para donde queremos ;)

```
Address   Value     Comment
00127898  004214CC  CALL to GetPrivateProfileStringA from tute_.004214CA
0012789C  004A6A90  Section = "REG"
001278A0  004A6A88  Key = "RegCode"
001278A4  004ACED4  Default = ""
001278A8  00127968  ReturnBuffer = 00127968
001278AC  00000028  BufSize = 28 (40.)
001278B0  004A6AA0  IniFileName = "mgutil_win.ini"
001278B4  004ACEF8  ASCII "-ÜH"
```

Ahora el buffer lo tiene en :00127968 que es donde almacenará el serial, si ahora retornamos con Ctrl+F9 caeremos en la zona de código culpable de todo esto:

```
Address   Hex dump        Disassembly                  Comment
0042148E  .  6A 28        PUSH 28
00421490  .  52           PUSH EDX
00421491  .  68 D4CE4A00  PUSH 004ACED4
00421496  .  68 946A4A00  PUSH 004A6A94                ASCII "UserName"
0042149B  .  68 906A4A00  PUSH 004A6A90                ASCII "REG"
004214A0  .  C68424 74860 MOV BYTE PTR [ESP+8674],0F
004214A8  .  FFD7         CALL EDI
004214AA  .  8BD8         MOV EBX,EAX
004214AC  .  68 A06A4A00  PUSH 004A6AA0                ASCII "mgutil_win.ini"
004214B1  .  8D8424 B8000 LEA EAX,DWORD PTR [ESP+B8]
004214B8  .  6A 28        PUSH 28
004214BA  .  50           PUSH EAX
004214BB  .  68 D4CE4A00  PUSH 004ACED4
004214C0  .  68 886A4A00  PUSH 004A6A88                ASCII "RegCode"
004214C5  .  68 906A4A00  PUSH 004A6A90                ASCII "REG"
004214CA  .  FFD7         CALL EDI
004214CC  .  8D8C24 54010 LEA ECX,DWORD PTR [ESP+154]
004214D3  .  51           PUSH ECX
```

Esos "CALL EDI" son los que recuperan el nombre y serial del archivo .ini que estamos estudiando, un poco después llegamos a esta zona:

```
Address   Hex dump        Disassembly                  Comment
004214F9  .  8D8C24 54010 LEA ECX,DWORD PTR [ESP+154]
00421500  .  50           PUSH EAX                     Arg2 = 00127968 ASCII "121212"
00421501  .  51           PUSH ECX                     Arg1 = 00127A08 ASCII "SyXe'05"
00421502  .  8BCD         MOV ECX,EBP
00421504  .  E8 B7140000  CALL 004229C0                tute_.004229C0
00421509  .  85C0         TEST EAX,EAX
0042150B  .v 74 0F        JE SHORT 0042151C
0042150D  .  83FB 04      CMP EBX,4
00421510  .  C705 E4DB4A0 MOV DWORD PTR [4ADBE4],1
0042151A  .v 73 0A        JNB SHORT 00421526
0042151C  >  C705 E4DB4A0 MOV DWORD PTR [4ADBE4],0
00421526  >  8D9424 B4000 LEA EDX,DWORD PTR [ESP+B4]
```

Donde empuja el nombre y el serial a la pila y hace un call, si entro en este veo que comienza a tratar el nombre en un loop:

```
Address   Hex dump    Disassembly                    Comment
00422C84  .  2BF3     SUB ESI,EBX
00422C86  >  8A10     MOV DL,BYTE PTR [EAX]          coge char[nombre]
00422C88  .  84D2     TEST DL,DL
00422C8A  .v 7D 04    JGE SHORT 00422C90
00422C8C  .  F6DA     NEG DL
00422C8E  .  8810     MOV BYTE PTR [EAX],DL          encripta char
00422C90  >  8A1406   MOV DL,BYTE PTR [ESI+EAX]
00422C93  .  3210     XOR DL,BYTE PTR [EAX]          encripta char
00422C95  .  80FA 30  CMP DL,30
00422C98  .  8810     MOV BYTE PTR [EAX],DL          lo guarda
00422C9A  .v 7D 12    JGE SHORT 00422CAE
00422C9C  .  0FBED2   MOVSX EDX,DL
00422C9F  .  83FA 30  CMP EDX,30
00422CA2  .v 7D 0A    JGE SHORT 00422CAE
00422CA4  >  83C2 03  ADD EDX,3
00422CA7  .  83FA 30  CMP EDX,30
00422CAA  .  8810     MOV BYTE PTR [EAX],DL          re-encripta char
00422CAC  .^ 7C F6    JL SHORT 00422CA4
00422CAE  >  8A10     MOV DL,BYTE PTR [EAX]
00422CB0  .  80FA 39  CMP DL,39
00422CB3  .v 7E 12    JLE SHORT 00422CC7
00422CB5  .  0FBED2   MOVSX EDX,DL
00422CB8  .  83FA 39  CMP EDX,39
00422CBB  .v 7E 0A    JLE SHORT 00422CC7
00422CBD  >  83EA 03  SUB EDX,3
00422CC0  .  83FA 39  CMP EDX,39
00422CC3  .  8810     MOV BYTE PTR [EAX],DL          re-encripta char
00422CC5  .^ 7F F6    JG SHORT 00422CBD
00422CC7  >  40       INC EAX
00422CC8  .  49       DEC ECX
00422CC9  .^ 75 BB    JNZ SHORT 00422C86             hemos terminado?
00422CCB  >  8BB424 94000( MOV ESI,DWORD PTR [ESP+94]
00422CD2  .  8BC3     MOV EAX,EBX
00422CD4  >  8A10     MOV DL,BYTE PTR [EAX]          digito [nombre_encriptado]
00422CD6  .  8A1E     MOV BL,BYTE PTR [ESI]          digito [serial]
00422CD8  .  8ACA     MOV CL,DL
00422CDA  .  3AD3     CMP DL,BL                      los compara
00422CDC  .v 75 1E    JNZ SHORT 00422CFC
00422CDE  .  84C9     TEST CL,CL
00422CE0  .v 74 16    JE SHORT 00422CF8
00422CE2  .  8A50 01  MOV DL,BYTE PTR [EAX+1]
00422CE5  .  8A5E 01  MOV BL,BYTE PTR [ESI+1]
00422CE8  .  8ACA     MOV CL,DL
00422CEA  .  3AD3     CMP DL,BL
00422CEC  .v 75 0E    JNZ SHORT 00422CFC
00422CEE  .  83C0 02  ADD EAX,2
00422CF1  .  83C6 02  ADD ESI,2
00422CF4  .  84C9     TEST CL,CL
00422CF6  .^ 75 DC    JNZ SHORT 00422CD4
```

Lo que hace en ella es encriptar el nombre, bueno en realidad lo pasa a dígitos numéricos, veamos como lo deja tras pasarle todo el loop:

```
Address   Hex dump                                        ASCII
00127A08  32 32 32 39 37 38 30 00 00 00 00 00 00 00 00 00  2229780.........
00127A18  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00127A28  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00127A38  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00127A48  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00127A58  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00127A68  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00127A78  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Recordando que partíamos de la cadena SyXe'05 que era el nombre introducido, tras pasar por el bucle anterior queda como 2229780 que tiene igualmente 7 dígitos, luego en el segundo bucle que vemos compara esta cadena generada con 121212 que fue el serial ingresado, además a la salida del call en función del resultado de esta comparación se modifica la variable global [4ADBE4] pero si probamos este serial acompañando al nombre veremos que no es correcto, jeje.. una trampa puesta para que piquemos (y vaya si lo hemos hecho, jajaja…) lo que haremos será seguir traceando la rutina ignorando esta cadena por ahora no vaya a ser que luego sirva para algo ¿? Justo después se mete en otra llamada donde realiza más cositas:

```
00421530  .  E8 EB170000   CALL 00422D20
00421535  .  85C0          TEST EAX,EAX
00421537  .  0F85 5F020000 JNZ 0042179C
```

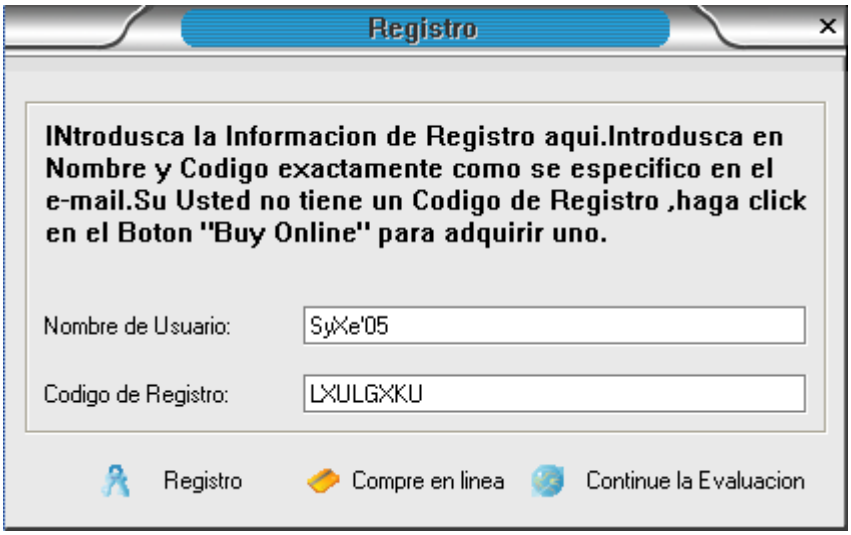
Dentro de esta llamada a :00422D20 vemos como trata con muchísimas otras cadenas sospechosas de este tipo:

```
004A6CD8  32 46 43 5A 43 43 42 32 2C 42 5A 4B 46 5A 47 31  2FCZCCB2,BZKFZG1
004A6CE8  32 2C 5A 5A 4D 45 43 32 33 47 2C 47 35 32 45 45  2,ZZMEC23G,G52EE
004A6CF8  32 46 55 2C 47 43 33 4D 33 55 46 46 2C 32 45 5A  2FU,GC3M3UFF,2EZ
004A6D08  45 43 48 32 4B 2C 42 34 5A 5A 5A 48 35 5A 2C 32  ECH2K,B4ZZZH5Z,2
004A6D18  42 42 55 46 46 33 4D 2C 43 46 47 43 46 45 46 45  BBUFF3M,CFGCFEFE
004A6D28  2C 35 5A 48 32 5A 46 31 43 2C 45 5A 5A 45 32 46  ,5ZH2ZF1C,EZZE2F
004A6D38  5A 31 2C 43 33 30 30 43 30 48 30 2C 45 46 5A 31  Z1,C300C0H0,EFZ1
004A6D48  47 32 5A 4D 2C 35 34 32 31 32 4D 55 5A 2C 32 48  G2ZM,54212MUZ,2H
004A6D58  48 46 31 45 33 42 2C 46 45 45 43 4D 46 35 32 2C  HF1E3B,FEECMF52,
```

Y las va comparando con nuestro serial en este bucle:

```
Address   Hex dump   Disassembly                 Comment
00422E51  .  52      PUSH EDX
00422E52  .  E8 318F0300  CALL 0045BD88          retorna serial en EAX
00422E57  .  8BBC24 142B0(  MOV EDI,DWORD PTR [ESP+2B14]  pone el nuestro en EDI
00422E5E  .  83C4 08 ADD ESP,8
00422E61  .  85C0    TEST EAX,EAX
00422E63  .v 74 4A   JE SHORT 00422EAF
00422E65  >  8BF7    MOV ESI,EDI
00422E67  >  8A10    MOV DL,BYTE PTR [EAX]       inicio loop comparador
00422E69  .  8A1E    MOV BL,BYTE PTR [ESI]       de seriales...
00422E6B  .  8ACA    MOV CL,DL
00422E6D  .  3AD3    CMP DL,BL
00422E6F  .v 75 1E   JNZ SHORT 00422E8F          salta => serial MALO ;(
00422E71  .  84C9    TEST CL,CL
00422E73  .v 74 16   JE SHORT 00422E8B           salta => serial VALIDO ;)
00422E75  .  8A50 01 MOV DL,BYTE PTR [EAX+1]
00422E78  .  8A5E 01 MOV BL,BYTE PTR [ESI+1]
00422E7B  .  8ACA    MOV CL,DL
00422E7D  .  3AD3    CMP DL,BL
00422E7F  .v 75 0E   JNZ SHORT 00422E8F
00422E81  .  83C0 02 ADD EAX,2
00422E84  .  83C6 02 ADD ESI,2
00422E87  .  84C9    TEST CL,CL
00422E89  .^ 75 DC   JNZ SHORT 00422E67
00422E8B  >  33C0    XOR EAX,EAX
00422E8D  .v EB 05   JMP SHORT 00422E94
00422E8F  >  1BC0    SBB EAX,EAX
00422E91  .  83D8 FF SBB EAX,-1
00422E94  >  85C0    TEST EAX,EAX
00422E96  .v 0F84 A601000(  JE 00423042
00422E9C  .  8D4424 0E  LEA EAX,DWORD PTR [ESP+E]
00422EA0  .  50      PUSH EAX
00422EA1  .  6A 00   PUSH 0
00422EA3  .  E8 E08E0300  CALL 0045BD88          retorna serial en EAX
00422EA8  .  83C4 08 ADD ESP,8
00422EAB  .  85C0    TEST EAX,EAX
00422EAD  .^ 75 B6   JNZ SHORT 00422E65          al inicio del loop!!
00422EAF  >  8D4C24 0E  LEA ECX,DWORD PTR [ESP+E]
00422EB3  .  8D5424 10  LEA EDX,DWORD PTR [ESP+10]
```

Hay otros 4 bucles más de estos donde se siguen comparando seriales del mismo tipo con nuestro 121212 del alma. Lo que hago es probar uno de ellos:



Acepto y sale el mensaje de antes diciendo que se debe reiniciar el programa y tal, pues nada lo hago y mi sorpresa al entrar sin la nagscreen inicial, además si ahora miro en el About me llevo otra sorpresa ;)

Allí dice claramente que quedamos registrados, está claro que es así, pues funciones que no funcionaban en modo Trial cuando el programa había expirado ahora sí funcionan.

En fin, una protección inexistente, se genera un serial que no hace servir para nada porque liego tiene cincuenta mil hardcoded serials, jejejeje…

Y hasta aquí llega el tutecito de hoy, una presa fácil para ir practicando. Un saludo y un abrazo para todo CracksLatinoS¡!!!!

Para cualquier cosa ya sabéis donde contactarme: syxe05@gmail.com XD